Predicting Fork-to-PR Conversion in Open-Source Repositories

Business Understanding

This phase focuses on understanding the project objectives from a business perspective and converting them into a data mining problem definition.

Determine Business Objectives

Background

Commercial context. Open-source software (OSS) organizations rarely sell the code itself. Revenue comes from the surrounding commercial layer: managed hosting, enterprise support, dual licensing, and premium features in an open-core model. In each case, the scale and engagement of the external developer community are central value drivers.

What a fork signals. A *fork* on GitHub can reflect one of two qualitatively different patterns:
Contributing use. The developer returns work to upstream through a pull request, which strengthens the project and the community.

Private downstream use. The fork is maintained on its own with no pull request to upstream: the project delivers value to the user but receives no contribution back.

Role of this project. The balance between these two patterns is not measured systematically today. *GH Archive* captures the public GitHub event stream and makes it possible to estimate fork-to-contribution ratios at repository scale. This project defines that measurement and trains a predictive model linking repository-level signals to each outcome.

Business Objectives

Primary objective. Determine, for a sampled set of GitHub repositories, what fraction of forks result in pull requests back to upstream, and identify which repository-level characteristics predict high or low contribution conversion.

Primary users of the analysis

- Open-source program managers who oversee community health and engagement strategy.
- Product strategy teams that need to understand how their open-source assets are being adopted and extended.
- Developer relations teams that prioritize outreach based on repository contribution potential.

Decision supported by this analysis. Understanding which types of repositories attract contributing forks versus passive downstream forks allows stakeholders to prioritize community investment, documentation effort, and contribution tooling for the repositories where upstream engagement is most likely.

Related business questions

- What fraction of forks for a given repository leads to at least one pull request within a defined period?
- What distinguishes repositories where forks tend to become contributions from those where they do not?
- Are there repository segments where conversion is consistently low despite high fork activity?

Business Success Criteria

The project outcome is considered successful if it produces:

- A reliable and reproducible repository-level fork-to-PR conversion metric computed from public event data.
- A predictive model that identifies high- and low-conversion repositories with better accuracy than a naive baseline.
- An interpretable set of repository features associated with conversion outcomes, reviewable by non-technical stakeholders.
- A documented pipeline that can be re-run on new time periods.

Subjective judgment of output usefulness rests with the Business Analyst and the Business Unit.

Assess Situation

Inventory of Resources

Personnel

- Business Unit.
- Project Manager.
- Data Scientist.
- ML Engineer.
- Business Analyst.

Data

- GH Archive public dataset in BigQuery (`githubarchive.month.*`).
- Core events: `ForkEvent`, `PullRequestEvent`.
- Context events for feature engineering: `PushEvent`, `WatchEvent`, `IssuesEvent`, `IssueCommentEvent`, `ReleaseEvent`.

Computing and software

- BigQuery for SQL extraction and aggregation.
- Python stack (pandas/polars, scikit-learn, Jupyter) for modeling and evaluation.

Requirements, Assumptions, and Constraints

Requirements

- Results must be interpretable via thresholds, feature rankings, or trend summaries.
- Results must be reproducible: identical inputs produce identical outputs; all steps versioned.
- Data is publicly available under open access; no legal restrictions on analytical use.
- Scope must be bounded: observation window, sampling frame, and label definition fixed before modeling.
- Outputs must include uncertainty ranges from validation folds.

Assumptions

- A fork is marked as converted when at least one PR is observed that can be reasonably matched to the fork within the chosen window. Due to the absence of explicit linkage, this relationship will be approximated using heuristic matching (e.g., user identity and temporal proximity).
- Multi-period sampling is more representative than a single-day snapshot.
- Repository-level aggregation reduces event-level noise.

Constraints

- **Fork-to-PR linkage is indirect.** The relationship must be inferred by matching repository identities across event types; no explicit link is stored in GH Archive.
- **Upstream metadata is sparse.** Static repository fields appear only when a pull request event is recorded. Repositories that never appear in such events must be described using event-aggregate features only.
- **Raw data volume is large.** One day of GH Archive is about 3 GB compressed. A three-month fork anchor plus a 90-day observation window spans roughly 270 GB compressed. All aggregation must run inside BigQuery before any export.
- **Class imbalance is expected.** Observed same-day fork-to-PR rate is 3–4%; even at 90 days the positive class will be a small minority.

Risks and Contingencies

- **Sampling bias across repository sizes.**
  - **Contingency:** Use stratified sampling by activity/popularity bands and report sample composition.
- **Noisy fork-to-PR matching.**
  - **Contingency:** Validate joins on sampled records and run consistency checks.
- **Temporal drift and seasonality.**
  - **Contingency:** Use time-based validation and compare multiple periods.
- **Over-interpretation of correlational results.**
  - **Contingency:** Report uncertainty and explicitly state non-causal interpretation.

Terminology

Business terminology

- **Fork**: a copy of a repository created by an external developer under their own account, typically to experiment or contribute.
- **Upstream repository**: the original repository from which a fork is derived.
- **Contribution**: an externally developed change submitted back to the upstream repository, typically as a pull request.

- **Contribution conversion**: the outcome where a fork leads to at least one pull request to upstream.
- **Fork-to-PR conversion rate**: the fraction of forks for a repository that resulted in at least one upstream pull request within a defined observation period.
- **Community health**: the degree to which a repository attracts active, contributing external developers.

Data mining terminology

- **ForkEvent**: a GH Archive event record that captures the creation of a fork. Used to identify fork instances and their upstream repositories.
- **PullRequestEvent**: a GH Archive event record that captures pull request actions. Used to detect whether a fork produced an upstream contribution.
- **Observation window**: the post-fork time interval within which a PR must appear for the fork to be labeled as converted. Example: 90 days after fork creation.
- **Repository-level aggregation**: grouping individual fork/PR event pairs by upstream repository to compute a conversion rate as a modeling target.
- **Stratified sampling**: selecting repositories proportionally across activity or popularity bands to ensure the sample is representative of the full distribution.

Costs and Benefits

Project costs

- **Data extraction:** design and execution of BigQuery SQL pipelines over multi-month GH Archive tables; estimated 2–4 major extraction runs with iterative refinement.
- **Label construction and validation:** fork-to-PR join logic, observation window tuning, and manual spot-checks on matched pairs to verify entity linkage correctness.
- **Feature engineering:** aggregation of event-based signals at the repository level (e.g., pushes, watches, issues) and construction of derived features (e.g., PR-to-fork ratios).
- **Modeling and evaluation:** training and cross-validating regression and classification baselines; hyperparameter search and ranking metric computation.
- **Reporting:** interpretation of model outputs, documentation of limitations, and production of final deliverables.
- **Infrastructure:** BigQuery query processing cost scales with the volume of scanned data; estimated low-to-moderate usage given partitioned queries and table wildcards.

Potential business benefits

- **Quantified community health signal:** for the first time, stakeholders obtain a concrete numeric measure—fork-to-PR conversion rate per repository—rather than relying on subjective community perception.
- **Prioritized portfolio view:** repositories can be ranked by expected contribution conversion, directing limited community investment toward projects with the highest return on engagement.

- **Early detection of passive-use segments:** repositories with many forks but low conversion are identified systematically, flagging potential gaps between what the project offers and what downstream users actually need.
- **Reusable pipeline:** the BigQuery extraction and feature construction logic can be re-run on any future period with no redesign, reducing the marginal cost of each subsequent analysis cycle.

Cost benefit comparison Under the assumptions below, expected quarterly benefit exceeds recurring quarterly operating cost and can recover setup cost within the first one to two quarters.

Rough quantitative estimate (illustrative assumptions)

- Repositories under active monitoring each quarter: 300.
- Average enterprise upgrade gross margin per converted repository: $40.
- If conversion risk is not detected early, expected margin impact on at-risk repositories: 10–15% (=$4–$6 per repository).
- The analysis gives one-quarter earlier warning for 25–35% of at-risk repositories.

Estimated quarterly benefit range

- $300 \times (0.25 \text{ to } 0.35) \times (\$4 \text{ to } \$6) \approx \$300$ to $630 per quarter.

Estimated project cost range

- Initial setup: 20 analyst hours.
- Quarterly refresh: 4 analyst hours.
- Assuming $20/hour internal cost: initial $400, then $80 per quarter.

Annualized benefit ($1,200–$2,520) is above annualized operating cost after setup ($320/year), and setup cost can be recovered within the first one to two quarters under these assumptions.

Determine Data Mining Goals

Data Mining Goals

- Build repository-level target variables from fork-to-PR conversion in fixed windows.
- Train either conversion-rate regression or bucketed conversion classification models.
- Produce ranking outputs for high vs. low expected conversion repositories.
- Quantify feature influence using interpretable model diagnostics.

Data Mining Success Criteria

- **Conversion rate prediction:** mean absolute error on held-out repositories must be lower than the MAE of the mean-prediction baseline; R^2 must be positive on the test fold.
- **Classification:** macro-averaged F1 score must exceed the stratified random baseline by at least 5 percentage points.
- **Ranking quality:** precision@10% must exceed the rate expected under random ranking; NDCG or lift@k reported for the top decile.
- **Validation stability:** performance metrics must not degrade by more than 15% between the best and worst time-based validation fold, confirming that results are not driven by a single time period.
- **Feature interpretability:** the top 5 features by importance must have a plausible directional relationship with conversion that can be explained to a non-technical audience without further analysis.
- **Sensitivity check:** conversion rates computed under two alternative observation windows (e.g., 30 days and 90 days) must yield qualitatively consistent rankings; large rank reversals flag a fragile label definition.

Produce Project Plan

Project Plan

Stage 1: Data understanding
Duration: 1 week.
Resources required: Data Scientist, Business Analyst, BigQuery.
Inputs: GH Archive day/month tables, event schemas.
Outputs: schema notes, feasibility checks, sampled repository list.
Dependencies: BigQuery access and agreement on the event types and time range to be queried.
Decisions made at this stage: anchor period (3 calendar months of `ForkEvent`); observation window candidates (30, 60, 90 days post-fork); stratified sampling bands (fork-count quintiles); minimum fork threshold per repository.

Stage 2: Data preparation
Duration: 2 weeks.
Resources required: Data Scientist, ML Engineer.
Inputs: sampled repositories, fork and PR event extracts.
Outputs: labeled repository-level dataset and feature table.
Dependencies: approved matching logic and observation windows.
Decisions made at this stage: final observation window (selected from candidates); feature groups — event-aggregate features (push rate, watch rate, issue rate, PR rate) and static metadata from `pull_request.base.repo` where available; train/test split boundary (chronological: first 2 months train, third month test).

Stage 3: Modeling
Duration: 2 weeks.
Resources required: Data Scientist, ML Engineer.
Inputs: training and validation datasets.
Outputs: baseline and tuned models, ranking outputs.

Dependencies: finalized feature set and validation protocol.
Decisions made at this stage: model candidates — mean-rate baseline, Logistic Regression, Random Forest, Gradient Boosting; target formulation — conversion rate regression or low/medium/high bucket classification; class imbalance handling via class weights and threshold tuning.

Stage 4: Evaluation
Duration: 1 week.
Resources required: Data Scientist, Business Analyst, Project Manager.
Inputs: trained models, hold-out predictions.
Outputs: metric report, error analysis, interpretation summary.
Dependencies: completed modeling runs and evaluation scripts.
Evaluation strategy: chronological hold-out on the third month; primary metrics MAE and R^2 for regression, macro-F1 for classification, precision@10% for ranking; sensitivity analysis over the two observation window lengths; SHAP values for top-5 feature interpretation.

Stage 5: Deployment
Duration: 1 week.
Resources required: ML Engineer, Project Manager.
Inputs: final SQL scripts, model artifacts, documentation.
Outputs: reproducible pipeline and project handover package.
Dependencies: approval of evaluation results.
Decisions made at this stage: pipeline delivered as versioned BigQuery SQL scripts plus a Python notebook; model serialised with `joblib`; final report documents all parameter choices and their rationale.

Review points and large-scale iterations

- Review point after Stage 1: verify sampling representativeness and schema assumptions.
- Review point after Stage 2: verify label quality and feature completeness.
- Iteration loop between Stage 2 and Stage 3 if label window or joins need adjustment.
- Review point after Stage 4: decide whether model quality is sufficient or another iteration is required.

Schedule-risk dependencies (actions)

- **Risk:** delayed BigQuery extraction due to oversized scans. **Action:** partitioned queries and narrower pilot windows first.
- **Risk:** weak target signal in selected period. **Action:** extend anchor period and rebalance sampled repositories.
- **Risk:** unstable metrics across folds. **Action:** enforce rolling time splits and simplify feature set.

Initial Assessment of Tools and Techniques

- **Data extraction:** BigQuery SQL with table wildcards and JSON extraction functions.
- **Transformation:** SQL aggregations plus pandas/polars for final feature shaping.
- **Modeling:** scikit-learn linear and tree-based baselines.

- **Evaluation:** time-based validation, ranking metrics, and calibration diagnostics.

Data Understanding

Collect Initial Data

The initial data for this project was obtained from the public GitHub Archive dataset available in Google BigQuery. This dataset contains a continuous stream of GitHub activity events (e.g., forks, pull requests, pushes, and issues) and is updated hourly.

Data Sources and Access

- **Source:** GitHub Archive (public dataset in BigQuery)
- **Access method:** SQL queries executed via Google BigQuery
- **Storage:**
  - Final datasets were directly constructed in BigQuery
  - Exported to Google Colab in `.parquet` format:
    - `activity_sample.parquet`
    - `fork_pr_sample.parquet`

Data Acquisition Process

The data collection process was implemented as a single-stage sampling pipeline directly on top of the raw GitHub Archive tables.

1. Extraction of event data from BigQuery for the year 2025
2. Filtering of relevant event types:
   - ForkEvent
   - PullRequestEvent
   - WatchEvent
   - PushEvent
   - IssuesEvent
   - IssueCommentEvent
   - ReleaseEvent
3. Construction of a sampled activity dataset (`activity_sample`), where:
   - repository-level activity metrics are aggregated
   - repositories are filtered by minimum fork activity
   - only active repositories (at least one interaction signal) are retained
   - a random sample of repositories is selected
4. Construction of a fork and pull request dataset (`fork_pr_sample`) using the same set of repositories to ensure consistency between features and target variables

Key Design Decisions

Several important design decisions were made to balance analytical goals and computational constraints:

- **Anchor periods:** Fork events are restricted to two 3-month windows (January–March and July–September), which define the reference periods for analysis
- **Observation window:** Pull request events are collected over extended seasonal windows (January–June and July–December) to allow for delayed contributions after forks
- **Minimum fork threshold:** Only repositories with more than 5 forks within the anchor period are retained to reduce noise from low-activity repositories
- **Activity filtering:** Repositories must exhibit at least one interaction signal (watches, pushes, issues, issue comments, or releases)
- **Sampling strategy:** A random sample of repositories is drawn after filtering to reduce data volume while preserving diversity
- **Seasonality handling:** Two distinct seasonal periods (winter–spring and summer–autumn) are used to reduce temporal bias and capture variation in developer activity

Problems Encountered and Solutions

- **Large data volume:** The GitHub Archive dataset is extremely large (tens of gigabytes for a single year). This was addressed by applying early filtering and sampling directly in BigQuery queries, avoiding materialization of full raw datasets.
- **Storage limitations:** The BigQuery sandbox imposes strict storage limits. This was mitigated by constructing sampled datasets directly, instead of storing intermediate full-scale tables.
- **High cost of payload field:** The `payload` field significantly increases data size [Figure 1]. It was excluded, and only structured fields (e.g., `repo.name`, `actor.login`) were used.
- **Sampling bias:** GitHub data is highly skewed toward popular repositories. This was mitigated by applying a minimum activity threshold and random sampling after filtering.
- **Temporal drift and seasonality:** Developer activity varies over time. This was addressed by explicitly defining two seasonal periods and aligning all features and targets with these periods.

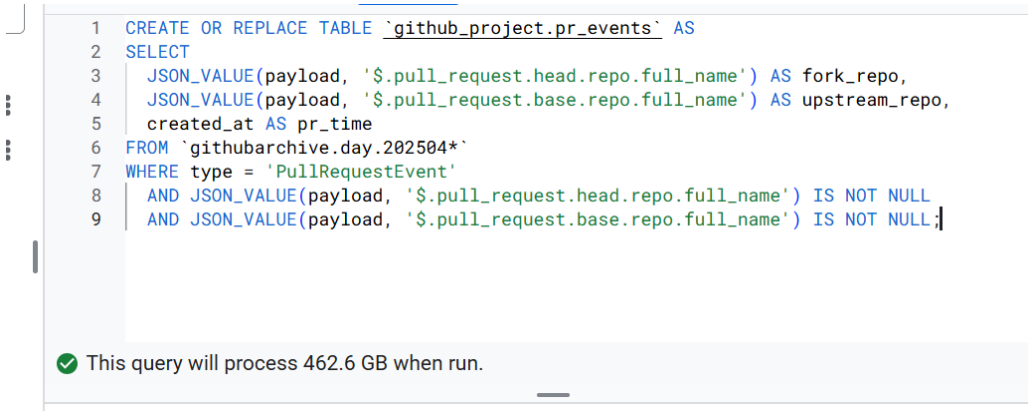


Figure 1: Influence of including the payload field on query size

Outcome

As a result of the data collection process, two main datasets were prepared:

- **activity_sample:** repository-level activity metrics, including watches, pushes, issues, issue comments, and releases
- **fork_pr_sample:** structured arrays of fork and pull request events with user identifiers and timestamps, aligned with the sampled repositories and seasonal periods

These datasets form the basis for subsequent data understanding, exploration, and preparation stages.

Describe Data

In this stage, the acquired datasets are examined at a surface level in order to understand their structure, size, and general properties. The goal is to assess whether the collected data is suitable for further analysis and modeling.

Data Format and Structure

Both datasets obtained during the data collection stage are stored in `.parquet` format and loaded into a tabular structure using pandas DataFrames. Each dataset represents repository-level observations, with rows corresponding to repository-season pairs.

The `activity_sample` dataset contains aggregated numerical metrics describing repository activity, while the `fork_pr_sample` dataset contains nested structures representing lists of events (forks and pull requests) for each repository.

The data types of the columns reflect this difference. In the activity dataset, numerical columns such as `watches`, `pushes`, `issues`, `issue_comments`, and `releases` are stored as integer types, while identifiers such as `repo_name` and `season` are stored as strings. In contrast, the fork/PR dataset stores event data in object-type columns, as they contain structured arrays of user and timestamp information.

Data Size and Dimensions

Both datasets contain 100,000 rows, corresponding to the sampled repository-season combinations.

The activity dataset [Figure 2] contains 9 columns:

- repo_name
- season
- repo_url
- public
- watches
- pushes
- issues
- issue_comments
- releases

	repo_name	season	repo_url	public	watches	pushes	issues	issue_comments	releases
0	GoodRequest/BackendAssignment-Fitness	winter_spring	https://api.github.com/repos/GoodRequest/Backe...	True	0	1	0	0	0
1	NREL/DE	winter_spring	https://api.github.com/repos/NREL/DE	True	0	1	0	0	0
2	danhpaiva/class-psc-algorithm-00	summer_autumn	https://api.github.com/repos/danhpaiva/class-p...	True	0	1	0	0	0
3	NaturalCoder/TI96	winter_spring	https://api.github.com/repos/NaturalCoder/TI96	True	0	25	0	0	0
4	josedom24/ic-html5	winter_spring	https://api.github.com/repos/josedom24/ic-html5	True	0	2	0	0	0

Figure 2: Head of activity table

The fork/PR dataset [Figure 3] contains 4 columns:

- repo_name
- season
- forks
- prs

	repo_name	season	forks	prs
0	Quitten/Autorize	summer_autumn	[{'event_time': 2025-09-13 12:19:55+00:00, 'ty...	[{'event_time': 2025-09-01 19:31:59+00:00, 'ty...
1	imzyb/MiSub	summer_autumn	[{'event_time': 2025-09-05 07:39:27+00:00, 'ty...	[{'event_time': 2025-11-01 07:20:49+00:00, 'ty...
2	facebookresearch/pytorch3d	summer_autumn	[{'event_time': 2025-09-22 18:15:37+00:00, 'ty...	[{'event_time': 2025-08-01 14:46:39+00:00, 'ty...
3	meltylabs/melty	summer_autumn	[{'event_time': 2025-08-17 20:25:43+00:00, 'ty...	[]
4	GouvernementFR/dsfr	summer_autumn	[{'event_time': 2025-09-01 14:44:37+00:00, 'ty...	[{'event_time': 2025-11-20 15:32:26+00:00, 'ty...

Figure 3: Head of fork_pr table

The memory footprint of the datasets is relatively small:

- Activity dataset: approximately 29 MB
- Fork/PR dataset: approximately 38 MB

This confirms that the data has been successfully reduced from the original large-scale source to a manageable size suitable for local analysis.

Field Description

The `activity_sample` dataset includes the following types of variables:

- **Identifiers:** repo_name, season
- **Metadata:** repo_url, public
- **Activity metrics:** watches, pushes, issues, issue_comments, releases

The `fork_pr_sample` dataset includes:

- **Identifiers:** repo_name, season
- **Event structures:**
  - forks: list of fork events (user identifier and timestamp)
  - prs: list of pull request events (user identifier and timestamp)

Thus, the datasets complement each other: one provides aggregated activity signals, while the other preserves detailed event-level information required for modeling contribution dynamics.

Initial Assessment

Overall, the acquired data satisfies the requirements for further analysis. The datasets are well-structured, compact, and contain both aggregated activity signals and detailed event-level information necessary for analyzing the relationship between repository activity and contribution behavior.

Explore Data

At this stage, exploratory data analysis was conducted to understand the distribution of key variables, relationships between features, and behavioral patterns relevant to the project objectives. The focus was placed on identifying signals that may influence the likelihood of fork-to-pull-request conversion.

Distribution of Activity Metrics

The distributions of repository activity metrics (`watches`, `pushes`, `issues`, `issue_comments`, `releases`) reveal a highly skewed structure.

While median values remain relatively low, the maximum values are extremely large, indicating the presence of a small number of highly active repositories dominating the distribution.

Outlier analysis confirms the existence of extreme repositories with disproportionately high activity across all metrics. This heavy right-skew is consistent across all activity features.

Implication: Logarithmic transformation or other scaling techniques will be required to stabilize variance and make patterns more detectable for modeling.

Seasonality Analysis

The dataset includes two seasonal segments: `winter_spring` (January–March) and `summer_autumn` (July–September).

The distribution across seasons is moderately imbalanced, but median activity values remain relatively consistent between the two periods across all metrics.

Implication: Seasonality does not significantly affect typical repository behavior, allowing both periods to be used jointly in modeling.

Public vs Private Repositories

All repositories in the dataset are marked as public.

Implication: The `public` attribute does not provide useful variation and can be excluded from the modeling stage.

Fork and Pull Request Distributions

The fork and pull request counts also exhibit strong skewness.

A substantial proportion of repositories receive forks but no pull requests:

- Approximately 34% of repositories have zero pull requests despite having forks

Implication: Conversion from fork to pull request is not guaranteed and represents a meaningful and non-trivial prediction target.

Repeated User Activity

Analysis of user behavior shows that most users fork a repository only once. However, pull request activity is significantly more intensive.

Repositories frequently contain users submitting multiple pull requests, indicating that contribution behavior is concentrated among a smaller number of highly active contributors.

Implication: Contribution dynamics are driven more by engaged contributors than by the number of casual users.

Relationship Between Activity and Contributions

Correlation analysis between activity features shows generally weak linear relationships[Figure 4]:

- The strongest correlations are observed between interaction-based metrics (e.g., issues and issue comments)
- Popularity metrics such as watches show very weak correlation with other features

```
pr_per_fork       1.000000
issue_comments    0.213205
pushes            0.137040
issues            0.082955
releases          0.071771
watches          -0.010566
```

Figure 4: Correlation with target

Further analysis of the relationship between activity and contribution outcomes shows:

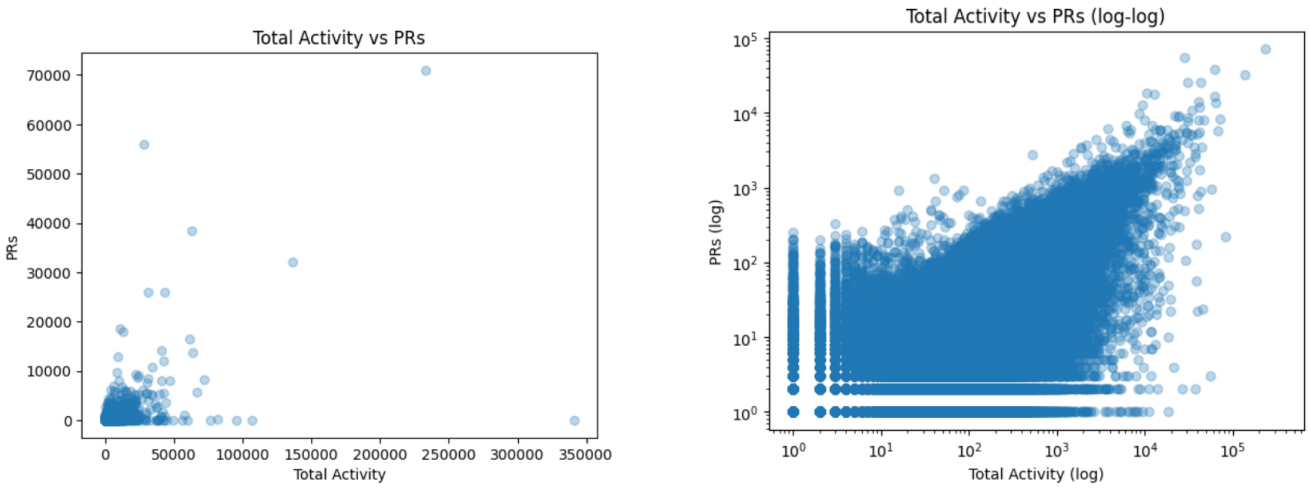
- `issue_comments` has the strongest positive association with conversion behavior
- `pushes`, `releases` and `issues` show weaker but still positive relationships
- `watches` (repository popularity) shows near-zero correlation with contribution outcomes

Implication: Interaction-based signals are more informative for predicting contributions than popularity-based metrics.

Log-Scale Analysis

Due to the strong skewness of the data, relationships were additionally analyzed in log scale.

Log-transformed scatter plots [Figure 5b] reveal clearer structure compared to raw-scale plots [Figure 5a], confirming that non-linear relationships exist between activity metrics and fork behavior.



(a) Raw scale

(b) Log scale

Figure 5: Activity vs PRs: raw vs log scale

Implication: Modeling approaches should account for non-linearity, either through transformations or non-linear models.

Derived Metric: PR per Fork

A derived metric was introduced to estimate repository-level conversion intensity:

$$\text{PR per Fork} = \frac{\text{PR count}}{\text{Fork count}}$$

The distribution of this metric is highly skewed:

- Median is low, indicating that most repositories have limited conversion
- Mean is significantly higher due to extreme outliers
- Maximum values are extremely large, driven by repositories with very few forks but many pull requests

Implication: While the metric captures conversion behavior, it is unstable for small fork counts and sensitive to extreme values. Careful handling (e.g., clipping or transformation) is required.

Key Findings

The exploratory analysis revealed several important characteristics of the data:

- All activity and contribution variables exhibit strong right-skew and extreme outliers
- A significant share of repositories does not convert forks into pull requests
- Contribution behavior is weakly correlated with activity metrics, indicating a non-trivial prediction task
- Interaction-based signals (especially issue discussions) are more predictive than popularity metrics
- Non-linear relationships are present, suggesting the need for transformation or non-linear modeling approaches

Verify Data Quality

In this stage, the quality of the collected datasets was assessed to identify potential issues that could affect downstream analysis and modeling. The evaluation focused on completeness, consistency, correctness, and the presence of anomalies.

Missing Values

Both datasets were examined for missing values across all columns.

No missing values were found in the `activity_sample` dataset. Similarly, the `fork_pr_sample` dataset contains no missing values at the top level.

To further validate completeness, nested event structures were inspected by expanding the `forks` and `prs` arrays.

The analysis shows:

- No missing user identifiers within fork events
- Missing entries in pull request arrays correspond to repositories with no pull requests rather than incomplete data

Implication: The dataset is complete; apparent missing values in nested structures reflect absence of events rather than data quality issues.

Data Consistency

Consistency checks were performed across datasets:

- No duplicate `repo_name{season` pairs were found in either dataset
- The set of repositories is consistent between `activity_sample` and `fork_pr_sample`

Implication: The datasets are properly aligned and can be safely joined without loss of information.

Validity of Values

All numerical activity metrics (`watches`, `pushes`, `issues`, `issue_comments`, `releases`) were verified to ensure they contain valid values.

- No negative values were detected across any metric

Time-based validation of fork events shows the expected temporal coverage:

- From: January 2025
- To: September 2025

This corresponds to the defined anchor periods used during data extraction.

Implication: The temporal filtering logic was applied correctly, and all values fall within expected ranges.

Outliers and Extreme Values

Quantile analysis [Figure 6b] confirms the presence of extreme values across all activity metrics.

Upper quantiles (e.g., 99th and 99.9th percentiles) are significantly higher than median values [Figure 6a], indicating heavy-tailed distributions.

Implication: These extreme values are not data errors but reflect the natural structure of GitHub activity, where a small number of repositories dominate engagement. However, they may disproportionately influence model training and will require appropriate handling.

	watches	pushes	issues	issue_comments	releases
count	1476.0	1476.0	1476.0	1476.0	1476.0
mean	2181.607724	2175.53252	424.816396	1960.968835	70.550813
std	4850.75688	10828.241334	1883.855765	7604.560614	165.27907
min	0.0	0.0	0.0	0.0	0.0
25%	42.0	101.0	14.0	56.0	1.0
50%	223.0	295.5	69.5	257.5	43.0
75%	3169.0	1047.0	263.5	1054.75	72.0
max	79049.0	340799.0	48592.0	199042.0	2143.0

(a) Summary statistics of activity metrics

	watches	pushes	issues	issue_comments	releases
0.950	400.0	269.05	80.0	263.0	7.0
0.990	1536.02	1192.02	301.0	1169.01	33.0
0.999	7814.032	7404.009	1376.029	6779.099	170.001

(b) Outliers in activity metrics

Figure 6: Distribution and extreme values of activity features

Feature Relationships and Redundancy

Additional validation was performed to assess relationships between features.

Correlation analysis shows generally weak linear relationships between activity metrics, with slightly stronger associations between interaction-based features (e.g., issues and issue comments).

No evidence of strong multicollinearity was observed.

Implication: Features provide complementary information and are suitable for use in modeling.

Target Stability Check

The derived metric `pr_per_fork` was analyzed to assess stability.

The distribution is highly skewed, with:

- Low median values
- High variance
- Extreme maximum values caused by small fork counts combined with multiple pull requests

Implication: The target variable is sensitive to outliers and may require transformation or alternative formulations during modeling.

Overall Assessment

The data demonstrates high overall quality:

- No missing values at the table level
- No duplicate records at the repository-season level
- Consistent structure across datasets
- Valid numerical and temporal values

The main characteristics identified are:

- Presence of extreme values (outliers)
- Highly skewed distributions
- Non-linear relationships between variables

These characteristics are expected in real-world GitHub data and do not represent data quality issues. The dataset is suitable for further analysis and modeling, provided that appropriate preprocessing techniques are applied.

Data Preparation

The data preparation phase covers all activities to construct the final dataset for modeling from the initial raw data. The implementation materializes two parallel modeling exports (30-day and 90-day label definitions) with the same feature side; the final choice of window for training belongs to the modeling phase, not to this step. Below, each step states *what* is done, *why* it is done, and *what* remains in the data after the step.

Select Data

Data selection during extraction

What was already fixed in the warehouse:

- **Schema focus:** the GitHub Archive extract keeps identifiers (repository, actor), event type and time, repository URL, visibility, and a derived `season` label; large or rarely needed structures (e.g. deep payload and most nested metadata) are excluded to limit size and to align rows across tables.

- **Event types:** the pipeline uses `ForkEvent` and `PullRequestEvent` for the conversion target, and `WatchEvent`, `PushEvent`, `IssuesEvent`, `IssueCommentEvent`, and `ReleaseEvent` for activity features, with the seasonal windows and repository filters set in the collection phase.
- **Why:** a narrow, consistent schema reduces noise, keeps joins predictable, and matches the business question (activity and fork-to-PR behaviour within defined horizons).

Data selection during EDA and modeling

Columns dropped from the modeling copy of the activity table.

- `public`: no usable variation in this sample, so it does not add signal and is removed to avoid a constant feature.
- `repo_url`: not used as a predictive input in the current design; the identifier `repo_name` suffices. The URL can be reattached later if static metadata is joined from outside.

What is retained.

- **Keys:** `repo_name` and `season` for every repository–season row from the sample.
- **Activity counters:** watches, pushes, issues, issue comments, releases (five nonnegative count variables).
- **Copy used in code:** `prep_activity` is the activity table after the drops above, so all subsequent cleaning and feature steps apply to this object only.

Row retention.

- No repository–season rows are removed at this step: the keys from the extract are kept.
- Alignment with labels is deferred to a later inner join, so any inconsistency is resolved explicitly rather than by silent row drops here.

Clean Data

Principle.

- Activity metrics show heavy tails (very popular repositories). These are treated as legitimate extremes, not as automatic row-level errors, so the strategy is tail reduction, not deletion of outlying repositories.
- Fork and PR event lists used for labels are not winsorized: labeling relies on raw timestamps and user linkage; only tabular activity counts are compressed for modeling stability.

Winsorization of activity counts. For each of the five activity counters:

1. Values are cast to `float64` before quantiles, so nullable integer types from Parquet do not break compatibility with real-valued quantile thresholds.
2. The 0.99 quantile is computed (fixed as `WINSOR_Q = 0.99` in the implementation).
3. Values above that quantile are capped; capped values are written to `*_winsor` columns; original columns are retained unchanged.

Outcome of cleaning. The modeling pipeline can use either raw or capped scales where appropriate; the full sample and event-level label inputs remain available without arbitrary row loss.

Construct Data

Sampling strategy

- **Unit of analysis:** one row = one repository in one season (not a single row aggregated across the whole year for the same repo).
- **Sample size:** defined upstream (e.g. cap and filters in the extract); this phase does not resubsample for class balance. The tables exported after merge are the full set of keys that pass join and label construction.
- **Why:** preserves the design of the initial draw and makes train/validation splits in modeling transparent relative to that population.

Matching forks to pull requests (label definition)

Conversion rule (per fork).

- A fork **converts** if there is at least one `PullRequestEvent` such that: (i) `user_login` matches the fork author, (ii) PR time is strictly after the fork time, (iii) PR time falls within a chosen observation window (in calendar days) after the fork.
- PRs on or before the fork, or after the end of the window, do not count toward conversion for that fork.

Two observation windows (not a single default). The implementation sets `OBSERVATION_WINDOWS_DAYS = (30, 90)` and runs the same labeling logic for each value. There is no single default observation length inside the data-preparation code: the notebook builds `labels_30d` and `labels_90d` and writes `fork_conversion_labels_obs30d` and `fork_conversion_labels_obs90d` (Parquet, or CSV if no Parquet engine is available). Each row includes `observation_window_days` (30 or 90).

Sensitivity of ranks between windows. A separate check compares repository-level `fork_conversion_rate` at 30 and 90 days (Spearman/Kendall correlation, decile reversals, and the share of 90-day conversion events not attributable within 30 days) and prints textual guidance for modeling (which file may be a reasonable first choice). That guidance is for the modeling step only; it does not change the fact that both full modeling tables are produced in Format Data.

Aggregates per repository and season. For each `repo_name` and `season`, the construction step records:

- `n_forks` — number of fork events in the cell;
- `n_converted_forks` — forks that meet the conversion rule within the chosen window;
- `fork_conversion_rate` = converted forks / fork count (this is not the same as total PR events divided by forks without user–time matching);
- `n_pr_events` — count of PR events in the list (for diagnostics and scale).

Derived features (activity side)

- For each of the five activity variables, log1p is applied to: (a) the raw count, and (b) the winsorized (`*_winsor`) value, producing `log1p_*` features suited to strong skew and nonnegative support.
- Original levels and `*_winsor` columns remain in the table so the analyst can choose scale or run robustness checks later.

Generated records

- **Grain:** one flat row per `repo_name` and `season`; nested fork/PR arrays are not written to these flat label files.
- **Empty PR lists:** rows are kept; conversion can be zero; such observations are not removed.
- **Only** fork–PR pairs that satisfy the match rules and fall inside the active window update `n_converted_forks` and the rate.

Integrate Data

Join logic.

- The prepared activity frame (`prep_activity`: identifiers, counts, winsorized and log-transformed fields) is merged with a label table on `repo_name` and `season`.
- The join is an inner merge with a one-to-one key check (`validate="one_to_one"`), and row counts are asserted equal to `prep_activity` and the label table. Each key appears at most once on each side and no duplicate fan-out is introduced.
- **Two integrated frames, not one "default":** the code builds `modeling_df_30d` and `modeling_df_90d` by merging to `labels_30d` and `labels_90d` respectively. Both are present for the same set of keys in the prepared sample; the notebook does not pick a single "main" target here—that choice is deferred to the modeling stage when loading `modeling_dataset_obs30d` or `modeling_dataset_obs90d`.
- **Result (per window):** one wide row per repository and season, combining feature columns with label fields (`observation_window_days`, `n_forks`, `n_converted_forks`, `fork_conversion_rate`, `n_pr_events`, and related fields from construction).

Format Data

Purpose. Stabilize column order, add convenient identifiers and optional stratification helpers, and export artifacts for the modeling phase.

Per-window merge and export pipeline. The export step applies, for each label table, an inner join of `prep_activity` to the chosen labels (same key logic as in Integrate Data), then:

- `repo_id`: integer code for `repo_name` via categorical codes; `repo_name` is retained.
- `has_conversion`: 1 if `n_converted_forks` > 0, else 0.
- `activity_strength`: sum of the five `log1p_` features on raw activity counts.
- `stratum_activity_quintile`, `stratum_fork_quintile`: quintile bins (0–4) from `pd.qcut` on `activity_strength` and on `n_forks` (`duplicates="drop"` on degenerate inputs), for optional stratified splits—they need not be used as model features.

- **Column order** in the exported file: meta (`repo_name`, `repo_id`, `season`), then all `log1p_*` columns (sorted by name in code), then the stratum block above, then the label block `observation_window_days`, `n_forks`, `n_converted_forks`, `fork_conversion_rate`, `n_pr_events`, `has_conversion`, then remaining columns (raw counts, `*_winsor`, etc.).
- **Shuffle:** the merged table is permuted with `sample(frac=1.0, random_state=335)` so the file order is not the extract order, while runs remain reproducible.

Output files.

- **Label-only files (one per window):** `fork_conversion_labels_obs30d` and `fork_conversion_labels_obs90d` in Parquet or CSV, as written in the construction step.
- **Full modeling tables (one per window):** `modeling_dataset_obs30d` and `modeling_dataset_obs90d` (Parquet, or `.csv` with the same stem if no Parquet engine is available). They share the same feature and metadata layout; only the label columns differ.